

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Программная инженерия
Дисциплина «Вычислительная математика»

Лабораторная работа №6
«Численное решение обыкновенных
дифференциальных уравнений»

Вариант 17

Выполнил

Храбров Артём Алексеевич

Проверил

Наумова Надежда Александровна

Санкт-Петербург, 2026

Цель работы

Решить задачу Коши для обыкновенных дифференциальных уравнений численными методами. Выполнить программную реализацию одношаговых и многошаговых методов, сравнить полученные приближенные решения с точным аналитическим решением, а также произвести оценку погрешностей вычислений.

1. Описание алгоритмов и рабочие формулы

Задача Коши для ОДУ первого порядка:

$$y' = f(x, y), \quad y(x_0) = y_0 \quad (1)$$

на интервале $[x_0, x_n]$ с заданным шагом h . Для решения задачи по варианту применяются следующие численные методы.

1.1 Метод Эйлера

$$y_{i+1} = y_i + hf(x_i, y_i) \quad (2)$$

1.2 Усовершенствованный метод Эйлера (с пересчетом)

$$y_{i+1} = y_i + \frac{h}{2} [f(x_i, y_i) + f(x_{i+1}, y_i + hf(x_i, y_i))] \quad (3)$$

1.3 Метод Адамса

Этап предиктора:

$$y_{i+1}^{(0)} = y_i + \frac{h}{24} (55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3}) \quad (4)$$

Этап корректора:

$$y_{i+1}^{(1)} = y_i + \frac{h}{24} (9f(x_{i+1}, y_{i+1}^{(0)}) + 19f_i - 5f_{i-1} + f_{i-2}) \quad (5)$$

Для старта метода первые 4 точки рассчитываются с помощью точного решения или одношагового метода высокого порядка.

1.4 Оценка погрешности

Для **одношаговых методов** оценка точности производится по правилу Рунге:

$$R = \frac{|y^h - y^{h/2}|}{2^p - 1} \quad (6)$$

где y^h — решение с шагом h , $y^{h/2}$ — решение с шагом $h/2$, p — порядок точности метода.

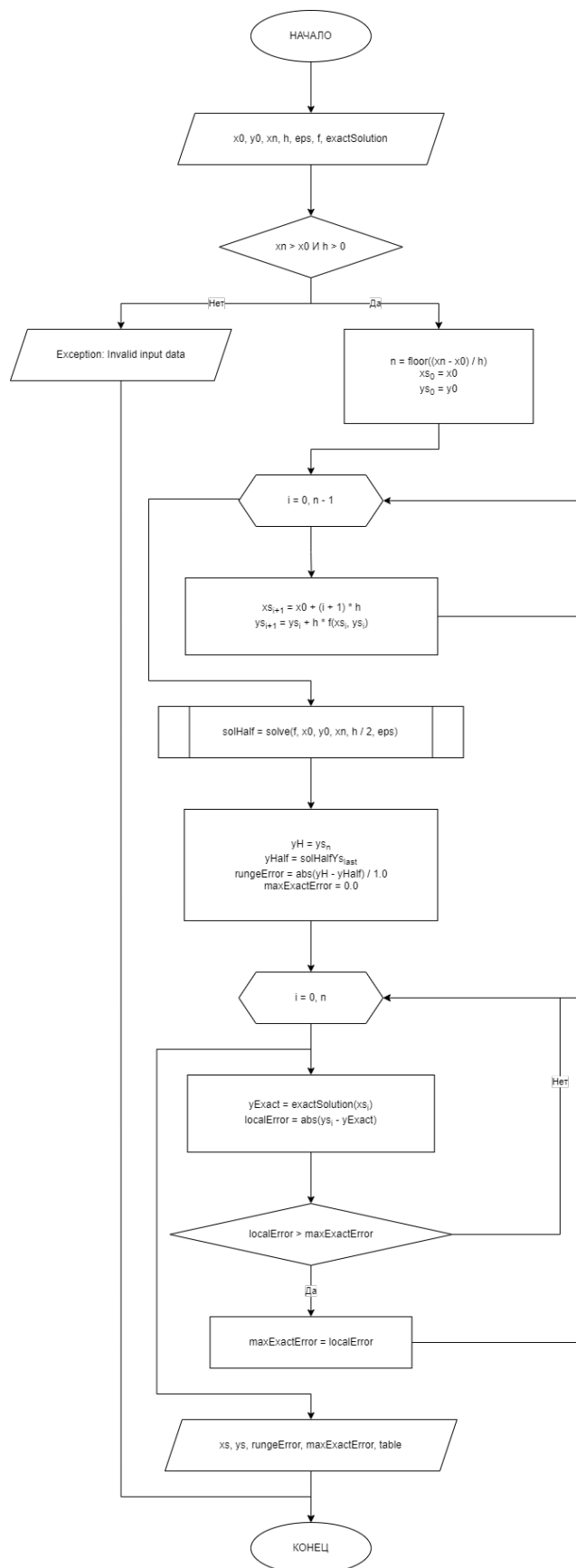
Для **многошагового метода** оценка точности производится путем сравнения с точным аналитическим решением задачи:

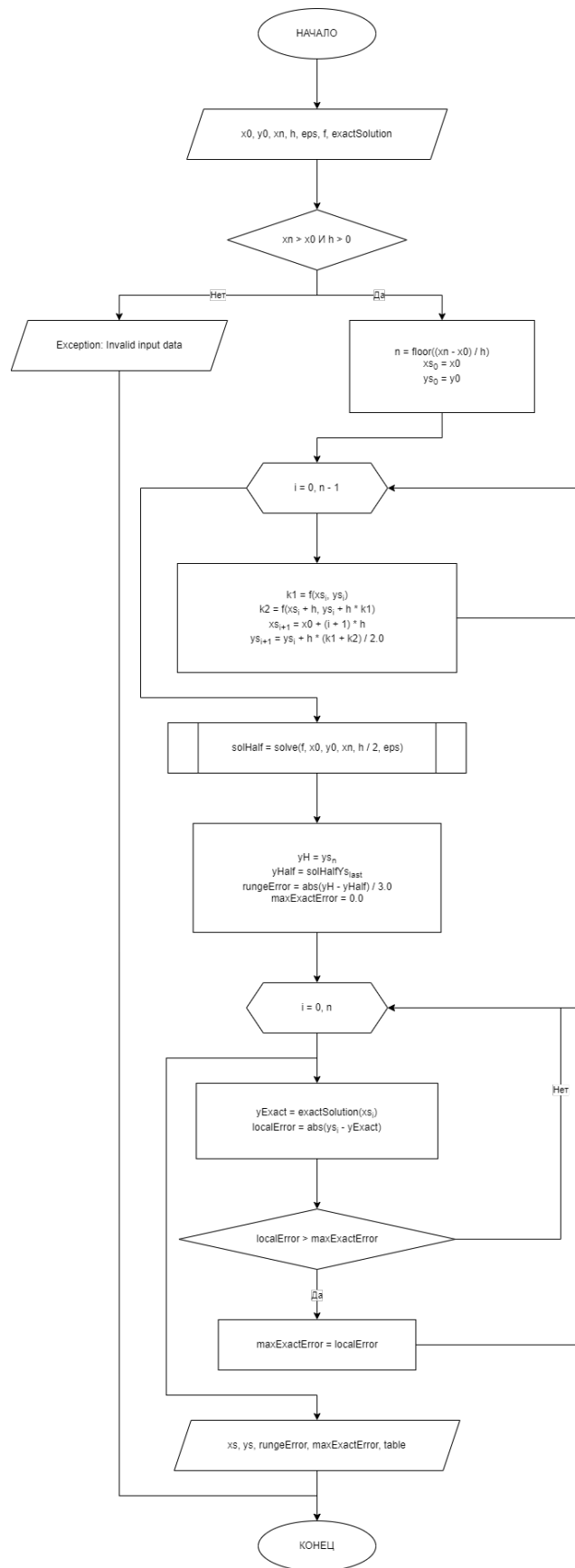
$$\varepsilon = \max_{0 \leq i \leq n} |y - y_i| \quad (7)$$

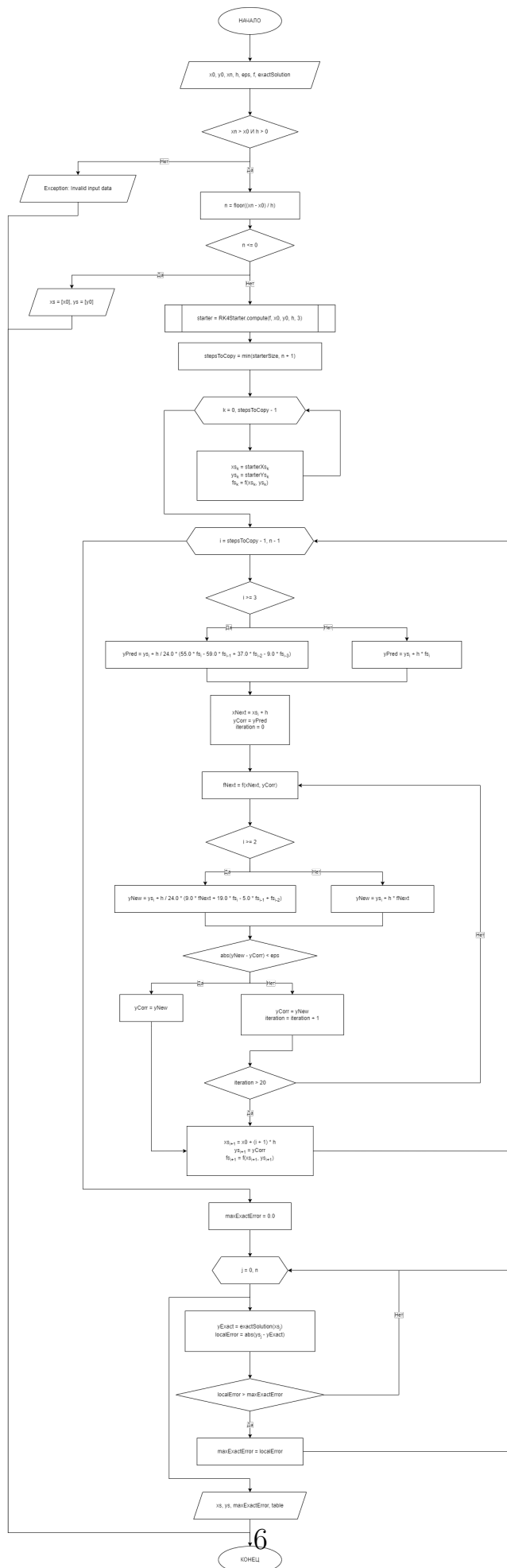
2. Блок-схемы алгоритмов

Метод

Эйлера





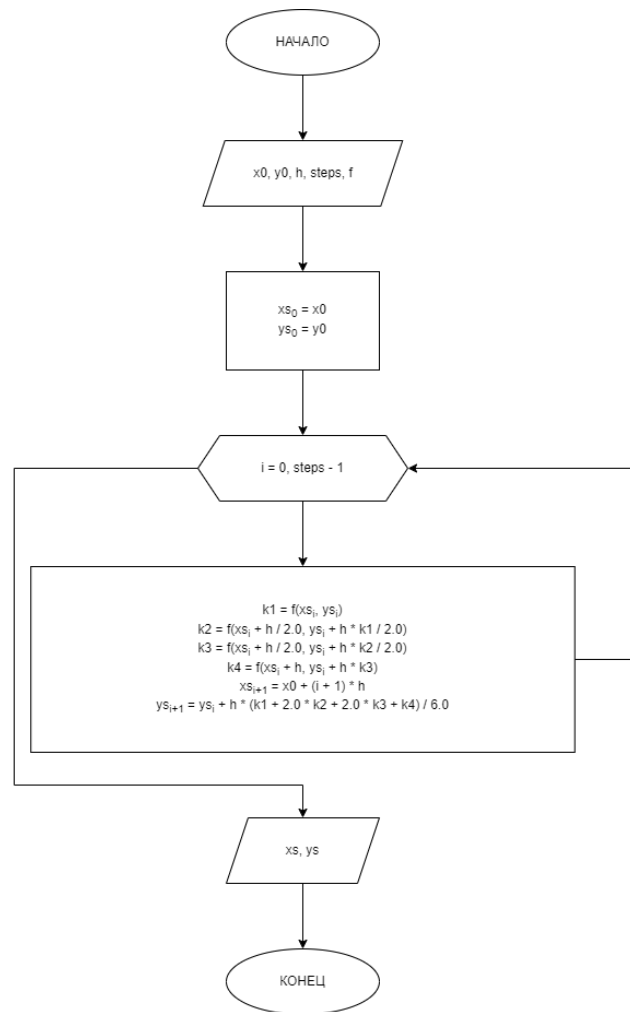


Генерация

точек

для

Адамса



3. Листинг программы

Исходный код программы доступен в репозитории:

[ссылка на GitHub](#)

```
1 package arhr.tech.comp_math_lab2.solver
2
3 import kotlin.math.floor
4 import org.springframework.stereotype.Component
5
6 @Component
7 class EulerSolver : ODESolver {
8     override val name: String = "Euler"
9     override val isOneStep: Boolean = true
10    override val order: Int = 1
11
12    override fun solve(f: ODEFunction, x0: Double, y0: Double, xn: Double, h
: Double, eps: Double?): ODESolution {
13        require(h > 0) { "h must be > 0" }
14        require(xn > x0) { "xn must be > x0" }
15
16        val n = floor((xn - x0) / h).toInt()
17        val xs = DoubleArray(n + 1)
18        val ys = DoubleArray(n + 1)
19        xs[0] = x0
20        ys[0] = y0
21        for (i in 0 until n) {
22            val xi = xs[i]
23            val yi = ys[i]
24            xs[i + 1] = x0 + (i + 1) * h
25            ys[i + 1] = yi + h * f(xi, yi)
26        }
27        return ODESolution(xs, ys)
28    }
29 }
30
31
32
33 package arhr.tech.comp_math_lab2.solver
34
35 import org.springframework.stereotype.Component
36 import kotlin.math.floor
37
38 @Component
39 class ImprovedEulerSolver : ODESolver {
40     override val name: String = "ImprovedEuler"
41     override val isOneStep: Boolean = true
42     override val order: Int = 2
```



```

43
44     override fun solve(f: ODEFunction, x0: Double, y0: Double, xn: Double, h
: Double, eps: Double?): ODESolution {
45         require(h > 0)
46         require(xn > x0)
47
48         val n = floor((xn - x0) / h).toInt()
49         val xs = DoubleArray(n + 1)
50         val ys = DoubleArray(n + 1)
51         xs[0] = x0
52         ys[0] = y0
53         for (i in 0 until n) {
54             val xi = xs[i]
55             val yi = ys[i]
56             val k1 = f(xi, yi)
57             val k2 = f(xi + h, yi + h * k1)
58             xs[i + 1] = x0 + (i + 1) * h
59             ys[i + 1] = yi + h * (k1 + k2) / 2.0
60         }
61         return ODESolution(xs, ys)
62     }
63 }
64
65
66 package arhr.tech.comp_math_lab2.solver
67
68 import kotlin.math.floor
69 import kotlin.math.min
70 import org.springframework.stereotype.Component
71
72 @Component
73 class AdamsSolver : ODESolver {
74     override val isOneStep: Boolean = false
75     override val order: Int = 4
76     override val name: String = "Adams(order=$order)"
77
78     override fun solve(f: ODEFunction, x0: Double, y0: Double, xn: Double, h
: Double, eps: Double?): ODESolution {
79         require(h > 0)
80         require(xn > x0)
81         val n = floor((xn - x0) / h).toInt()
82         if (n <= 0) return ODESolution(doubleArrayOf(x0), doubleArrayOf(y0))
83
84         val m = order
85         val starter = RK4Starter.compute(f, x0, y0, h, m - 1)
86         val xs = DoubleArray(n + 1)
87         val ys = DoubleArray(n + 1)
88         val stepsToCopy = min(starter.xs.size, n + 1)

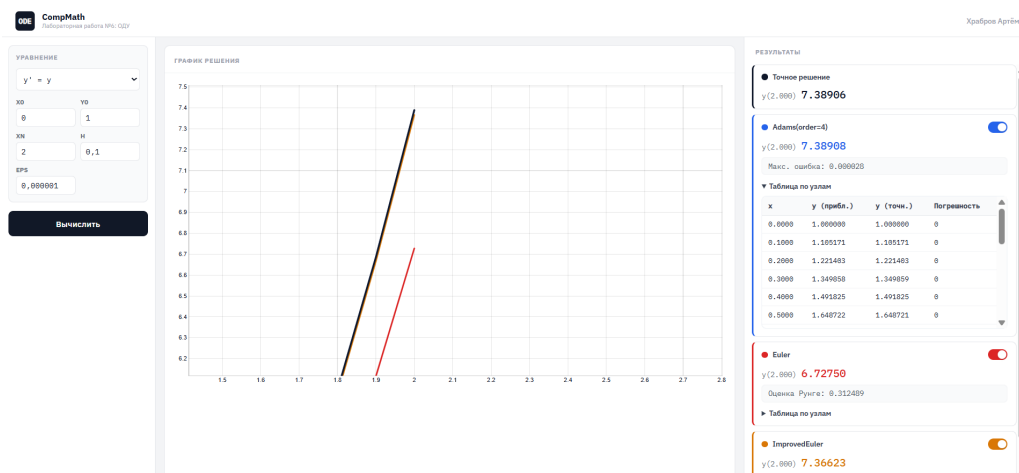
```

```

89     for (i in 0 until stepsToCopy) {
90         xs[i] = starter.xs[i]
91         ys[i] = starter.ys[i]
92     }
93     val epsLocal = eps ?: 1e-9
94
95     val fs = DoubleArray(n + 1)
96     for (i in 0 until stepsToCopy) fs[i] = f(xs[i], ys[i])
97
98     for (i in stepsToCopy - 1 until n) {
99         val yPred = if (i >= 3) {
100             ys[i] + h / 24.0 * (55.0 * fs[i] - 59.0 * fs[i - 1] + 37.0 *
fs[i - 2] - 9.0 * fs[i - 3])
101         } else {
102             ys[i] + h * fs[i]
103         }
104
105         val xNext = xs[i] + h
106         var yCorr = yPred
107         var iteration = 0
108         while (true) {
109             val fNext = f(xNext, yCorr)
110             val yNew = if (i >= 2) {
111                 ys[i] + h / 24.0 * (9.0 * fNext + 19.0 * fs[i] - 5.0 *
fs[i - 1] + fs[i - 2])
112             } else {
113                 ys[i] + h * fNext
114             }
115             if (kotlin.math.abs(yNew - yCorr) < epsLocal) {
116                 yCorr = yNew
117                 break
118             }
119             yCorr = yNew
120             if (++iteration > 20) break
121         }
122
123         xs[i + 1] = x0 + (i + 1) * h
124         ys[i + 1] = yCorr
125         fs[i + 1] = f(xs[i + 1], ys[i + 1])
126     }
127
128     return ODESolution(xs, ys)
129 }
130 }

```

4. Пример работы программы



Вывод

В ходе выполнения лабораторной работы я программно реализовал одношаговые методы (Эйлер, усовершенствованный Эйлер) и многошаговый метод Адамса.

Анализ результатов и графиков показал, что классический метод Эйлера дает наибольшую погрешность и является самым грубым. Усовершенствованный метод Эйлера и метод Адамса показывают высокую степень совпадения с точным аналитическим решением.